

Search, Constraint Satisfaction, Swarms and Sequential Decisions: Four AI Methods on Four Dhaka Problems

Your Full Name
your.email@example.com

July 14, 2026

Abstract. This report covers three Artificial Intelligence laboratory assignments, the third of which has two parts. In Assignment 1 we route a traveller across the real OpenStreetMap road graph of Dhaka (28,094 intersections, 70,197 segments) under a twelve-factor cost model, and compare three uninformed and three informed search algorithms against six heuristics. In Assignment 2 we model a citywide fuel shortage as a constraint optimisation problem and compare five solvers, including forward checking with MRV and a min-conflicts local search. Assignment 3 asks for a problem that a single weak agent cannot solve but a population can. Part A places Wi-Fi access points on a university hall floor with a particle swarm optimiser written from scratch; the key result is an ablation showing that thirty particles which do not communicate are statistically no better than random search, while the same thirty sharing a single number gain 2.47 points of coverage. Part B models the hall's water tank under Dhaka load-shedding as a Markov decision process and compares Value Iteration (model-based) against Q-learning (model-free). Our main finding there is negative and, we think, the more useful one: a roughly-right model is worth more than 82 simulated years of experience, and even a fourfold error in the model still beats Q-learning. Model-free learning earns its place only when the transition structure cannot be written down at all.

Keywords: Artificial Intelligence; heuristic search; constraint satisfaction; particle swarm optimisation; Markov decision processes; reinforcement learning

1. Introduction

All four problems in this report are set in Dhaka. That is deliberate. Each assignment asks for a different class of AI method, and the easiest way to understand why a method exists is to point it at a problem it was built for and then at one it was not.

Assignment 1 is a search problem: the world is a graph, the answer is a path, and the only question is how much of the graph you have to look at before you are sure. Assignment 2 is a constraint problem: there is no path, only an assignment, and the difficulty is that the constraints interact. Assignment 3 is neither. The search space becomes continuous and the objective stops being differentiable, so we cannot enumerate and we cannot descend a gradient. That is the gap population-based methods fill. And in its second part the world stops being static: the environment answers back, stochastically, and the question becomes not "what is the best plan" but "what is the best rule for acting when I do not know what happens next".

We wrote every algorithm in this report from scratch. No DEAP, no PySwarm, no Gym, no stable-baselines. We wanted to be able to explain every line in a viva, and a library would have hidden most of them.

The rest of the report follows the assignments in order. Section 2 reviews the relevant literature, Sections 3–5 present each assignment with its formulation, method and results, and Section 6 says what we learned and where we think the work is weak.

2. Literature Review

2.1. Assignment 1: Heuristic search on a weighted graph

Shortest-path search on a weighted graph is settled ground. Dijkstra's algorithm [3] finds the optimal path by expanding nodes in order of accumulated cost, and A* [9] adds an admissible heuristic $h(n)$ that never overestimates the remaining cost, so the optimum is preserved while fewer nodes are expanded. Weighted A* [18] inflates the heuristic by a factor $w > 1$ and trades a bounded loss of optimality (at most w times the optimal cost) for speed. The textbook treatment is Russell and Norvig [20], Chapter 3.

What is not settled is what the edge weights should be. Almost all published routing work minimises distance or estimated travel time. Neither captures what a person in Dhaka is actually optimising, which may include water-logging, street lighting after dark, or whether a road is safe for a woman travelling alone at 10pm. Our contribution in this assignment is not a new algorithm; it is a cost model, and an empirical study of how classical algorithms behave once the cost stops being a metric anyone has tuned. The road graph itself comes from OpenStreetMap via OSMnx [2].

2.2. Assignment 2: Constraint satisfaction and optimisation

A CSP is a triple of variables, domains and constraints, and the standard complete solver is backtracking search with propagation and ordering heuristics [20]. Arc consistency (AC-3) [15] prunes domain values that cannot participate in any solution. Forward checking [8] is the cheaper, incremental cousin: after each assignment it removes now-inconsistent values from the domains of unassigned neighbours, and fails early when a domain empties. The two dominant ordering heuristics are Minimum Remaining Values (assign the most constrained variable first, so that failure surfaces near the top of the tree) and Least Constraining Value (try the value that rules out the fewest options for everyone else).

Complete search guarantees a solution if one exists, but returns nothing if one does not. Local search trades that guarantee away. Min-conflicts [16] starts from a complete but broken assignment and repeatedly repairs the most conflicted variable, and it famously solves the million-queens problem in seconds. It cannot prove unsatisfiability, but it degrades gracefully, which is exactly what a fuel-allocation system needs when there is genuinely not enough fuel.

2.3. Assignment 3, Part A: Population-based search

When the objective is non-convex, non-differentiable and expensive, gradient methods have no gradient to follow and exact methods have too much space to enumerate. Population methods sample the space with many candidate solutions at once and let information flow between them. Genetic algorithms [6, 10] do this by recombining chromosomes; ant colony optimisation [4] does it by depositing pheromone on good partial paths; particle swarm optimisation [14] does it by having each candidate accelerate towards both its own best-known position and the best position found by anyone. Shi and Eberhart [21] added the inertia weight w that lets the swarm start broad and finish fine, and that is the form taught in the course and used here.

The theoretical backstop is the No Free Lunch theorem [26]: averaged over all possible objective functions, every optimiser performs identically. So the interesting question is never "is PSO good", it is "is PSO good *on this landscape*, and can I show it". We took that seriously enough to build the ablation in Section 5.3 that tries to kill our own claim.

2.4. Assignment 3, Part B: Decision making under uncertainty

A Markov decision process is a set of states, actions, a transition kernel $P(s' | s, a)$ and a reward [1, 11]. Value Iteration repeatedly applies the Bellman optimality operator, which is a γ -contraction, so it converges to the unique optimal value function V^* at a geometric rate. It is

model-based: the sum over s' is only computable because somebody handed you P .

Q-learning [24, 25] throws the model away. It samples one transition at a time and bootstraps off its own current estimate of the best next action, and it provably converges to Q^* under the Robbins–Monro conditions on the learning rate and infinite visitation of every state-action pair [13, 23]. Even-Dar and Mansour [5] showed that the step-size schedule matters a great deal in practice: a polynomial rate $\alpha_n = 1/n^\omega$ with $\omega \in (0.5, 1]$ converges, while a constant α converges only to a neighbourhood of Q^* . Sutton and Barto [22] is the standard reference for all of this.

Pump scheduling under uncertainty is an active application area [7, 12, 17], though that work uses deep RL on large water-distribution networks and is chasing scale. Our problem is deliberately small enough that Value Iteration can compute the exact optimum, which is what lets us measure every other method’s regret against ground truth instead of against another approximation.

2.5. Literature Review Summary and Experimental Focus

The reviewed literature shows AI problems being attacked through systematic search, constraint propagation, population-based stochastic search, and learning from interaction, with the choice driven by the structure of the problem rather than by any method’s inherent superiority [20, 26].

Our experimental focus follows from that. In each assignment we do not simply run the algorithm the unit was about; we run it against the strongest honest alternative we can build, at a matched budget, and report where it loses. Assignment 1 measures nodes expanded and solution cost against a Uniform-Cost baseline that is optimal by construction. Assignment 2 measures backtracks, nodes and objective against basic backtracking with no heuristics at all. Assignment 3A measures fitness against random search and grid search at an identical number of fitness evaluations. Assignment 3B measures policy regret against the exact optimum computed by Value Iteration, which is available precisely because we kept the state space small.

3. Assignment 1: Multi-Factor Route Search Across Dhaka

3.1. Problem Formulation

The input is the drivable road network of central Dhaka, taken from OpenStreetMap for the bounding box (90.34, 23.70) to (90.46, 23.86), reduced to its largest strongly connected component: 28,094 intersections, 70,197 directed road segments, about 5,826 km of road. The input also includes a source node, a goal node, and a *traveller context*: gender, social setting, age band, vehicle type, time of day, and weather.

The output is a path. The objective is to minimise

$$C(\mathcal{P}) = \sum_{e \in \mathcal{P}} \text{length}(e) \cdot w_{\text{len}} \cdot \prod_k m_k(e, \text{ctx}), \quad (1)$$

where each m_k is a multiplier for one factor: road condition, traffic, risk, personal safety, lighting, water-logging, crime, gender and social context, vehicle-highway suitability, age, weather, and street width. Multipliers are amplified by the context, so the same edge costs different amounts to different travellers at different hours.

The honest limitation, stated up front: every one of those edge attributes is *synthetic*. They are generated deterministically from OSM tags and coordinates with a fixed seed. They are plausible, they are reproducible, and they are not measured. This assignment therefore demonstrates algorithmic behaviour on a realistic-*shaped* cost surface; it does not claim to give real routing advice for Dhaka.

3.2. Methodology

We implemented six search algorithms over the same cost function: BFS, DFS and Uniform-Cost Search (uninformed), and Greedy Best-First, A* and Weighted A* with $w = 1.8$ (informed).

Note that BFS and DFS are run *against the realistic cost*, not against hop count, so their solution cost is directly comparable to the others.

We implemented six heuristics of varying quality. Two are provably admissible. The tighter one, `network_relaxed`, is

$$h(n) = \text{shortest_road_length}(n \rightarrow \text{goal}) \times \text{best_possible_cost_per_metre}(\text{ctx}), \quad (2)$$

where the first factor is a reverse Dijkstra on raw physical length and the second walks every cost factor to its most favourable achievable value. It is admissible because every real edge costs at least $\text{length}(e) \times \text{best_per_metre}$, and the shortest road length lower-bounds the length of any specific path. The reverse Dijkstra is computed once per goal and cached, so it is $O(V \log V)$ once and $O(1)$ thereafter.

The sweep is 50 source–destination pairs (1.2–12km apart) $\times$ 6 algorithms $\times$ 6 heuristics $\times$ 7 traveller contexts, giving 7,350 logged runs. We record nodes expanded, nodes generated, revisits, maximum frontier size, path cost, effective branching factor (solved by Newton iteration), and runtime.

3.3. Results and Discussion

Table 1 gives the medians. Three things are worth pulling out.

Table 1: Assignment 1, median over all runs. Cost is the multi-factor objective, not metres.

Algorithm	Nodes expanded	Path cost	Runtime (ms)	Success
UCS (Dijkstra)	14,145	212,375	95.0	100%
A*	13,362	212,375	129.3	100%
Weighted A* ($w=1.8$)	12,858	212,375	121.9	100%
BFS	14,693	312,343	20.6	100%
DFS	14,169	2,401,773	—	88%
Greedy Best-First	169	293,218	1.2	100%

First, optimality behaves exactly as the theory says. UCS, A* and Weighted A* all return a median cost of 212,375, identical to the last decimal. That is the check that the heuristics really are admissible: an inadmissible one would have shown up here as a cheaper- looking but actually worse path. Weighted A* returning the same cost is luck rather than law (its bound only guarantees within $1.8\times$), but it is reassuring.

Second, Greedy Best-First is the most interesting row in the table. It expands 169 nodes where A* expands 13,362 — roughly 79 times fewer — and runs in 1.2ms against 129ms. And it pays for it: its median path costs 293,218, about 38% worse than optimal. This is the speed-versus-optimality trade in one line, and it is the row we would point a first-year student at.

Third, DFS is a disaster, and usefully so. Its median path cost is 2.4 million, eleven times optimal, its median depth is 1,292 edges against A*'s 128, and it fails outright on 42 of 350 runs because it hits the depth cap. On a graph with 28,000 nodes and cycles everywhere, depth-first search wanders.

The heuristic sweep (Figure 1) is quieter than we expected. Every heuristic gives A* the same optimal cost, and the spread in nodes expanded is only 12,658 (`context_aware`) to 14,145 (`zero`, i.e. plain UCS). That is an 11% reduction for our best heuristic, which is disappointing. The reason is visible in the data: our admissible heuristic is very loose. It predicts about

159,000 against an actual cost around 167,000 on the pairs we checked — it captures only a few percent of the true cost, because `best_possible_cost_per_metre` has to assume every factor is simultaneously at its best, and on a twelve-factor multiplicative model that bound is extremely weak. A tighter admissible heuristic on a multiplicative cost model is the obvious next piece of work.

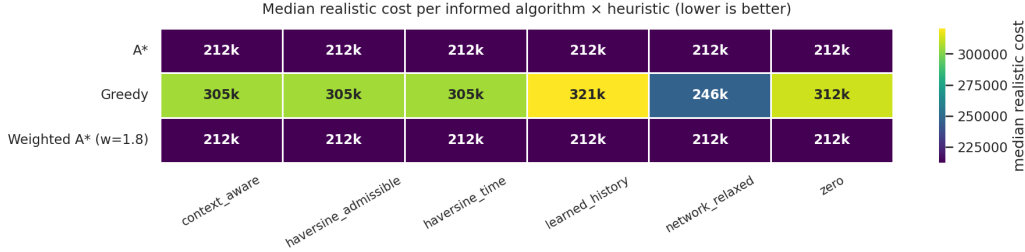


Figure 1: Nodes expanded for each (algorithm, heuristic) pair. The informed algorithms separate strongly from the uninformed ones; the heuristics separate only weakly from each other, because our admissible bound is loose.

4. Assignment 2: Fuel-Crisis Allocation as a Constraint Optimisation Problem

4.1. Problem Formulation

A city is short of fuel. Stations hold limited reserves of petrol, diesel and octane; a heterogeneous fleet of ambulances, buses, trucks, cars and motorbikes needs refuelling. If every vehicle drives to its nearest station, the ambulances get stuck behind the motorbikes and stations run dry of the wrong fuel.

We model it as a constraint *optimisation* problem rather than a pure CSP, on the principle that a system which returns nothing when the problem is infeasible is useless in a crisis. Variables $X = \{x_1, \dots, x_N\}$, one per vehicle. The domain D_i of x_i is the set of feasible (station, pump, time-slot) triples. Hard constraints: fuel-type compatibility; reachability within the vehicle’s remaining range; *pump exclusivity* (no two vehicles at the same pump in the same slot); station supply capacity per fuel type; and operating-hour windows. The soft objective, to be minimised, is

$$J(S) = w_{\text{dist}} \cdot \text{distance} + w_{\text{wait}} \cdot \text{wait} + w_{\text{prio}} \cdot \text{priority penalty} + w_{\text{unassigned}} \cdot |\text{unassigned}|, \quad (3)$$

with $w_{\text{unassigned}} = 100$. That last term is what turns the CSP into a COP: leaving a vehicle unserved is expensive but legal, so the solver always returns its best partial assignment.

4.2. Methodology

Five solvers: basic backtracking with no heuristics; backtracking + MRV; backtracking + LCV; backtracking + forward checking + MRV + degree tie-break (with AC-3 as a one-shot preprocess); and min-conflicts local search. Sizes $N \in \{10, 20, 30, 40, 50\}$, three seeds each, a 1.8 s wall-clock cap per run, and a 2,000-step budget for min-conflicts. We record nodes expanded, backtracks, constraint checks, runtime, the final objective J , and the fraction of vehicles left unassigned.

4.3. Results and Discussion

The main effect is the combinatorial blow-up. Basic backtracking expands 20.7 nodes at $N = 10$ and 220,763 at $N = 50$, roughly four orders of magnitude for a five-fold increase in problem size, and at $N = 50$ it never terminates on its own — every seed is pinned at the 1.8 s cap with a success rate of 0. The composite solver (FC + MRV + degree) expands 15,880 nodes at the same size, a 93% reduction, and it is the only complete solver that still finds a full assignment some of the time (Table 2).

Table 2: Assignment 2 at the largest size, $N = 50$, mean over three seeds. Lower J is better.

Solver	Nodes	Backtracks	Objective J	Unassigned
Basic backtracking	220,763	220,728	3,542.0	29.3%
Backtracking + MRV	91,025	90,996	4,439.5	40.0%
Backtracking + LCV	18,891	18,863	4,188.9	43.3%
FC + MRV + degree	15,880	15,833	1,718.4	6.7%
Min-conflicts	1,890 [†]	0	1,927.8	8.0%

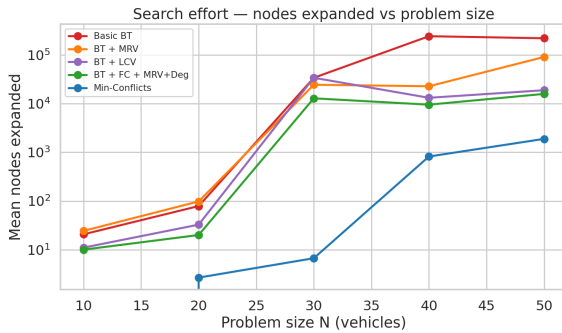
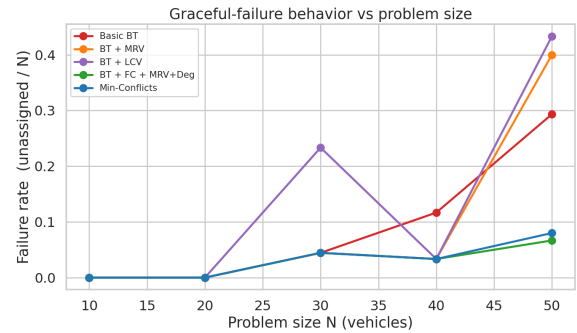
[†] Repair steps, not tree nodes. Min-conflicts is not a tree search.

Two results deserve comment because they are not what a naive reading of the textbook predicts.

Min-conflicts performs exactly **zero** backtracks at every problem size. That is not a bug and it is not an achievement — it is definitional. Min-conflicts never builds a partial assignment, so it has nothing to unwind. It starts complete and broken, and repairs. We flag it because "backtracks" is on our metrics list and reporting a zero without explaining it would look like a missing measurement.

The more awkward result is that plain MRV is *worse* than basic backtracking on the objective at $N = 50$ (4,439 against 3,542) despite expanding four times fewer nodes. This looked wrong until we looked at the unassigned column: MRV leaves 40% of vehicles unplaced against basic backtracking's 29%. Since $w_{\text{unassigned}} = 100$ dominates J , cross-solver comparisons of J at large N are really comparisons of how many vehicles got dropped, and are confounded. Runtime at $N \geq 40$ is similarly not a measure of time-to-solve; it is a measure of our time cap. We should have separated "quality of the assignment made" from "how much of the problem was attempted", and we did not.

On the easy sizes the composite solver is essentially perfect: at $N = 10$ and $N = 20$ it expands exactly N nodes with 0 backtracks — one node per variable, no wrong turns at all. Every solver achieves 100% success at $N \leq 20$; the phase transition to partial assignments begins at $N = 30$.

(a) Nodes expanded vs N (log scale).

(b) Fraction of vehicles left unassigned.

Figure 2: Assignment 2. Propagation flattens the exponential; graceful degradation is what the COP formulation buys.

5. Assignment 3: Population-Based Search and Sequential Decision Making

This assignment has two parts. Part A requires a problem that a single weak agent cannot solve but that a population of them can. Part B requires a problem for Value Iteration and Q-learning. We chose two separate problems rather than forcing one problem to serve both, but they share a

setting: a University of Dhaka residential hall. Part A places its Wi-Fi; Part B runs its water pump.

5.1. Part A — Problem Formulation: Wi-Fi access-point placement

The hall has a tight budget of K access points for one floor. Rooms far from an AP, or shadowed by a concrete partition, get an unusable signal. We choose the continuous positions of the K APs to maximise occupancy-weighted room coverage.

Decision variable: $\mathbf{x} = (\mathbf{a}_1, \dots, \mathbf{a}_K) \in \mathbb{R}^{2K}$, the AP coordinates. Received signal at room i from AP j follows a log-distance indoor path-loss model with discrete wall attenuation [19]:

$$\text{RSSI}_{ij} = P_{\text{tx}} - (L_0 + 10 n \log_{10} d_{ij}) - \gamma c_{ij}, \quad (4)$$

where $c_{ij} \in \mathbb{Z}_{\geq 0}$ counts the concrete partitions the line of sight crosses. Each room is served by its strongest AP, softened by a logistic so that signal *margin* is rewarded, and the objective is

$$\max_{\mathbf{x}} \quad F(\mathbf{x}) = 100 \cdot \underbrace{\frac{\sum_i w_i \rho_i(\mathbf{x})}{\sum_i w_i}}_{\text{weighted coverage \%}} - \lambda_{\text{sep}} \sum_{j < k} [\max(0, d_{\min} - \|\mathbf{a}_j - \mathbf{a}_k\|)]^2, \quad (5)$$

subject to the APs lying inside the floor. The penalty term enforces a minimum separation (co-channel interference).

Why this is not a gradient problem: F contains a $\max_j$ (a kink), a logistic threshold, and above all the *integer* wall-crossing count c_{ij} , which makes F piecewise-constant in $\mathbf{x}$. So $\nabla F = 0$ almost everywhere and is undefined on the kinks. Exact enumeration is no better: the space is continuous, and discretising it throws away the optimum, which is exactly what we demonstrate below.

Instance: a 60×40 m hall block, 40 rooms with occupancy weights, 3 concrete partitions, $K = 3$ APs (so a 6-dimensional search), $P_{\text{tx}} = 20$ dBm, $n = 3.3$, $\gamma = 8$ dB, threshold -66 dBm, $d_{\min} = 10$ m.

5.2. Part A — Methodology

We implemented the inertia-weight PSO of Shi and Eberhart [21], which is the form on the course slides:

$$\mathbf{v} \leftarrow w \mathbf{v} + c_1 r_1 (\mathbf{p}_{\text{best}} - \mathbf{x}) + c_2 r_2 (\mathbf{g}_{\text{best}} - \mathbf{x}), \quad (6)$$

$$\mathbf{x} \leftarrow \mathbf{x} + \mathbf{v}. \quad (7)$$

Each particle *is* a complete candidate layout. The inertia w decays linearly from 0.9 to 0.4, moving the swarm from exploration to exploitation; velocities are clamped to 20% of each axis range to stop the swarm exploding; positions are clamped to the floor with the offending velocity component zeroed, which is constraint repair by an absorbing wall. Swarm size 30, 100 iterations, $c_1 = c_2 = 1.49445$, 15 seeded runs.

The comparison is budget-matched, and we enforce it with an assertion at runtime rather than trusting ourselves: one PSO run costs exactly $P(T+1) = 30 \times 101 = 3030$ fitness evaluations, and Random Search is handed exactly 3030 samples. Grid Search lays a 5×5 lattice of candidate sites and evaluates all $\binom{25}{3} = 2300$ combinations, which fits inside the budget with room to spare, so it is not truncated.

5.3. Part A — Results and Discussion

Under an identical 3,030-evaluation budget, PSO reaches a mean weighted coverage of 89.91% against 87.43% for Random Search and 89.46% for exhaustive Grid Search (Table 3). The

gain over random is 2.47 points, or 2.83% in relative terms, and it is significant by a one-sided Wilcoxon signed-rank test over the 15 paired seeds ($W = 120.0$, $p = 3.05 \times 10^{-5}$). PSO is also steadier: its run-to-run standard deviation is 0.26 against random's 0.43, so its variance is about a third of random's.

Table 3: Assignment 3A. Identical fitness-evaluation budget. Hard coverage is the percentage of rooms actually above the usable-link threshold.

Method	Evals	Best	Mean	Std	Hard coverage
PSO (ours)	3030	90.01	89.91	0.26	100.0%
Random Search	3030	88.29	87.43	0.43	97.5%
Grid Search	2300	89.46	89.46	0.00	100.0%

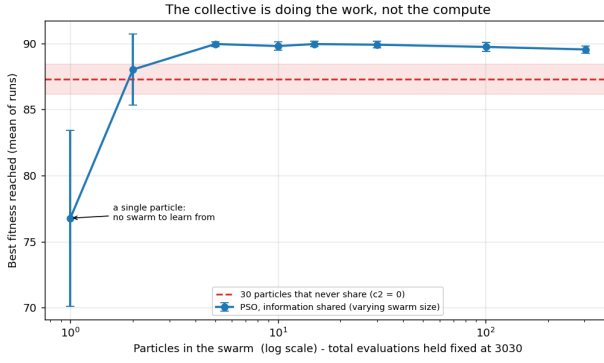
The Grid Search row is the one we care about, because it took two attempts to get right. In our first instance the rooms sat on a perfect rectangular grid, and Grid Search *beat* PSO (89.19 against 88.41). That result was correct and it was our fault: if the demand points are perfectly gridded, then grid-aligned AP positions are exactly where the optimum lives, and we had handed the discrete method the answer. Real rooms are not perfectly gridded. We added positional jitter ($\sigma = 1.5$ m) so that the optimum sits *off* any fixed lattice, which is both more realistic and the whole reason to use a continuous optimiser. After that change PSO wins. We report the earlier failure because it is the sharpest thing we learned in this assignment: a discrete baseline can look strong for reasons that have nothing to do with the algorithm.

Is the swarm actually doing anything? The assignment's real requirement is that individually weak agents solve collectively what none can solve alone. That is a claim, so we tested it, with two ablations at the identical 3,030-evaluation budget (Figure 3).

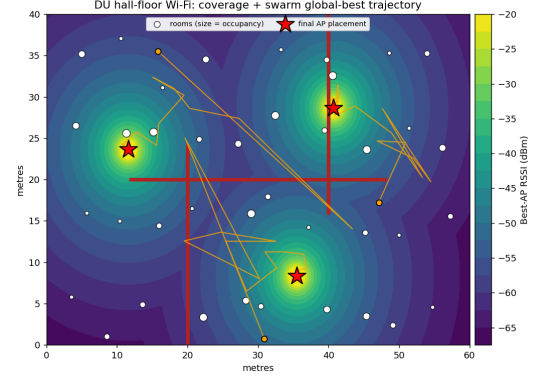
Shrink the swarm to a *single* particle and give it all 3,030 evaluations to itself. With one particle, $\mathbf{p}_{\text{best}}$ and $\mathbf{g}_{\text{best}}$ are the same point, the social term vanishes identically, and PSO degenerates into one lone agent with momentum and a memory. It scores 76.78, with a standard deviation of 6.66: not only much worse, but wildly unreliable from run to run. This is the single weak agent the assignment asks us to beat.

The sharper experiment keeps all thirty particles and simply sets $c_2 = 0$. Same number of agents, same budget, same everything; the particles still search and still remember their own best. They just never tell each other anything. They score **87.30 $\pm$ 1.15** — which is statistically indistinguishable from Random Search's 87.43 ± 0.43 .

That is the result we would defend the assignment on. Thirty independent searchers with no communication are worth no more than throwing 3,030 darts. Restore one number — the single shared $\mathbf{g}_{\text{best}}$ — and the same thirty agents, at the same cost, gain 2.61 points of fitness and connect every room in the hall instead of stranding one. The population is not what helps. The communication is.



(a) The ablation. Budget held fixed throughout.



(b) Coverage under the best layout, with the global-best trajectory.

Figure 3: Assignment 3A. Left: one particle scores 76.8; thirty silent particles score 87.3, no better than random; thirty communicating particles score 89.9. Right: the swarm’s global-best APs migrating to their final positions (red stars) around the concrete partitions.

The swarm-size curve also shows the expected U-shape. At a fixed budget, too few particles means no information to share; too many means too few iterations to use it. Five to fifteen particles is the sweet spot here (89.95), and 303 particles — which leaves only 10 iterations — falls back to 89.55.

5.4. Part B — Problem Formulation: the hall water tank under load-shedding

The same hall keeps its water in an overhead tank fed by an electric pump. Three things make this a decision problem and not a plumbing one. The grid fails, and it fails hardest in the evening, when the hall wants water. Evening outages are also long: once the power goes it tends to stay gone. And the hall owns a diesel generator that can drive the pump through an outage, but it buys a fixed ration of diesel each morning — burn it at 7am on a hunch and there is none left at 9pm.

Formally, an infinite-horizon discounted MDP. State $s = (L, t, g, F)$ with tank level $L \in \{0..10\}$ (units of 100 L), hour $t \in \{0..23\}$ (wrapping; this is a continuing task), grid up $g \in \{0, 1\}$, and diesel remaining $F \in \{0, 1, 2\}$. That is $11 \times 24 \times 2 \times 3 = 1584$ states. Actions are $\{\text{idle, pump on grid, pump on diesel}\}$, restricted by a state-dependent availability mask: pumping on grid is not offered during an outage (the switch does nothing) and diesel is not offered when the drum is empty.

Demand is a truncated Poisson whose mean depends on the hour, with peaks at 06:00–09:00 and 18:00–22:00. The grid is a two-state Markov chain whose failure and restore probabilities depend on the hour; in the evening, failure probability is 0.35 and restore is only 0.15, so an evening outage lasts about 6.7 hours in expectation. The reward is

$$r = -(\text{pump cost} + c_{\text{overflow}} \cdot \text{spill} + c_{\text{shortage}} \cdot U), \quad (8)$$

with $c_{\text{shortage}} = 20$ per unit of water students wanted and did not get, against a grid pump-hour cost of 1 and a diesel pump-hour cost of 6. We use $\gamma = 0.99$: at one hour per step that is an effective horizon of 100 hours, comfortably longer than the 24-hour cycle the whole problem turns on.

One subtlety that matters. The realised shortage U depends on the realised demand D , and D is *not* recoverable from s' — once the tank hits zero you cannot tell how much demand went unmet. So r is not a function of (s, a, s') . This is fine, and it is the standard disturbance form of

an MDP: $r = r(s, a, w)$ with $w = (D, g')$. Value Iteration only ever needs the expected reward $R(s, a) = \mathbb{E}_w[r]$, which is what we tabulate, and the Bellman operator is still a γ -contraction. Q-learning consumes the *realised* r . It has to: if we fed it $\mathbb{E}[U]$ we would have leaked the demand distribution into the "model-free" agent, and the entire distinction the assignment is about would collapse.

5.5. Part B — Methodology

We built the MDP with two faces that must never disagree. `build_model()` returns the explicit sparse $P(s' | s, a)$ and $R(s, a)$ that Value Iteration consumes. `step()` returns one sampled (s', r) and no probability ever leaves it; that is the only thing Q-learning is allowed to call. A unit test Monte-Carlo-estimates P and R back out of `step()` over 8,000 samples per state-action pair and asserts they match the table within sampling error. Without that gate, the classic silent failure is that the two faces disagree about, say, whether the pump acts before or after demand, Q-learning converges to a different optimum, and you spend a week blaming the learning rate.

Value Iteration sweeps $V_{k+1}(s) = \max_{a \in A(s)} [R(s, a) + \gamma \sum_{s'} P(s' | s, a) V_k(s')]$ to a residual of 10^{-9} .

Q-learning runs ε -greedy on the simulator with the update $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$. We use exploring starts (a uniformly random state each episode), because without them large parts of the state space are never visited under a decent policy and the argmax there is meaningless. The learning rate is $\alpha_n = 1/(1 + n(s, a))^{0.7}$, which satisfies Robbins–Monro [5]. Episodes are 24 hours, but the task is continuing, so we bootstrap normally at the cut and never zero the terminal value.

Every policy in this section — Value Iteration’s, Q-learning’s, and the baselines’ — is scored the same way: by *exactly* solving the linear system $(I - \gamma P_\pi)V^\pi = R_\pi$ under the *true* model. This removes Monte-Carlo evaluation noise from the comparison entirely. Policies are ranked by their exact expected return, not by which one drew a luckier rollout.

Baselines. The *myopic* policy is Value Iteration with $\gamma = 0$: the formally correct greedy rule, not a strawman. The *caretaker* policy is a two-threshold reactive rule (pump on grid below level T_g , run diesel below level T_d) with the *same action set* as the optimal policy, and we grid-search both thresholds and report its best version. Beating a crippled baseline would prove nothing.

We also added a baseline that could sink our whole thesis, which we will come to.

5.6. Part B — Results and Discussion

Value Iteration behaves exactly as the theory says. It converges in 2,273 sweeps (0.23 s), and the Bellman residual decays at an empirical rate of **0.9900** — which is γ to four decimal places, as the contraction mapping theorem requires (Figure 4a). Independently, V^* from iterating the optimality operator agrees with V^{π^*} from directly solving the linear system to 1.8×10^{-8} . Two different routes, one answer.

The problem genuinely needs lookahead. This had to be checked, because if a greedy rule were nearly optimal the assignment would be vacuous. It is not. The myopic $\gamma = 0$ policy loses **121%**, and even the best tuned reactive threshold rule — which has the generator, and whose thresholds we optimised for it — still loses **14.5%** (Table 4).

Table 4: Assignment 3B. Exact expected return under the true model; rollout statistics over 1,000 simulated days.

Policy	Exact V	Regret	Cost/day	Shortage units/day
Value Iteration (optimal)	-163.6	0%	32.8	0.78
Tuned caretaker rule	-187.4	14.5%	36.9	0.94
Myopic greedy ($\gamma = 0$)	-361.0	120.6%	78.1	3.08
Random (legal actions)	-697.5	326.2%	166.3	6.85
Always idle	-2507.8	1432.5%	527.7	26.38

We can point at the anticipation directly. At 14:00 with a 4/10 tank and the grid up, pumping costs 0.97 *more* than idling this hour, and the optimal policy pumps anyway, because the action is worth +4.69 overall. Of that advantage, +5.66 comes purely from the future. The agent is paying now for water it will need after the lights go out. Figure 5 shows this structurally: the region where the optimal policy burns diesel expands sharply once the evening window opens.

The main experiment, and the answer we did not want. The obvious story to tell is "Dhaka's published load-shedding schedule is optimistic, so a planner using it will be wrong, so use model-free Q-learning instead." We set out to demonstrate that, and the data refused.

We gave Value Iteration a *deliberately wrong* model, scaling the believed grid-failure probability by a factor and then scoring the resulting policy in the real hall (Figure 4b). Value Iteration turned out to be far less sensitive to this than we expected. Believing outages are twice as rare as they are costs only 1.5% regret. Four times as rare costs 2.7%. You have to tell the planner the grid *never* fails before its regret (21.4%) exceeds Q-learning's.

Meanwhile Q-learning, after 720,000 environment steps (that is 82 simulated *years* of hourly hall operation), sits at **15.1% $\pm$ 2.4** regret, with 100% coverage of the legal state-action space. It is converging, but slowly, and $\gamma = 0.99$ is part of why: a long effective horizon and a high-variance reward (a shortage costs 20 in a single hour) is a punishing combination for a tabular method.

Then we added the arm that could falsify us. Take the *same* transitions Q-learning consumed, count them into a maximum-likelihood estimate $\hat{P}(s'|s, a) = N(s, a, s')/N(s, a)$, and run Value Iteration on that. Same samples, two different ways of spending them. This *certainty-equivalence* planner reaches **1.4% $\pm$ 0.2** regret, ten times better than the Q-learner trained on identical data.

But it does *not* beat every prior. A planner who is only mildly wrong — believing outages are $1.25\times$ or $0.75\times$ their true rate — scores 0.11% and 0.24%, which is better than the model we learned from 720,000 samples. Certainty-equivalence only overtakes the prior once the prior is *badly* wrong (at or beyond a factor of two in either direction). We had written the opposite in an earlier draft, and the table we printed directly underneath it said so; the corrected statement is the more interesting one.

Ranking everything by regret makes the picture plain:

Table 5: Everything, ranked. Regret against the exact optimum, under the true model.

	What it knows	Regret
Value Iteration, correct model	the exact P	0%
Value Iteration, mildly wrong model	P off by $1.25\times$	0.1%
	P off by $0.75\times$	0.2%
Certainty-equivalence VI	$\hat{P}$ from 720k samples	1.4%
Value Iteration, badly wrong model	P off by $4\times$	2.7%
	P off by $10\times$	6.7%
Tuned caretaker threshold rule	nothing; two tuned numbers	14.5%
Q-learning	720k samples, no model	15.1%
Value Iteration, no outage model	believes the grid never fails	21.4%
Myopic greedy ($\gamma = 0$)	the model, but no future	120.6%

So the honest conclusion inverts the one we went looking for:

- A roughly-right model is worth more than 82 simulated years of experience. That is the result we did not expect and it is the one we would lead with.
- Value Iteration degrades gracefully. A planner with a fourfold error in its outage estimate (2.7%) still comfortably beats a Q-learner given 720,000 steps (15.1%).
- Our Q-learner also loses to the tuned two-parameter threshold rule (14.5%), which does no learning at all. We would rather state that than have it extracted from us.
- Therefore the case for model-free learning here is *not* "the model might be wrong" — it can be quite wrong and still win. It is narrower and it is real: model-free is what you reach for when you cannot write down the transition structure at all. Ours has 1,584 states and we could write it down, which makes this an honest advertisement for Value Iteration and a poor one for Q-learning. Knowing why is the point.

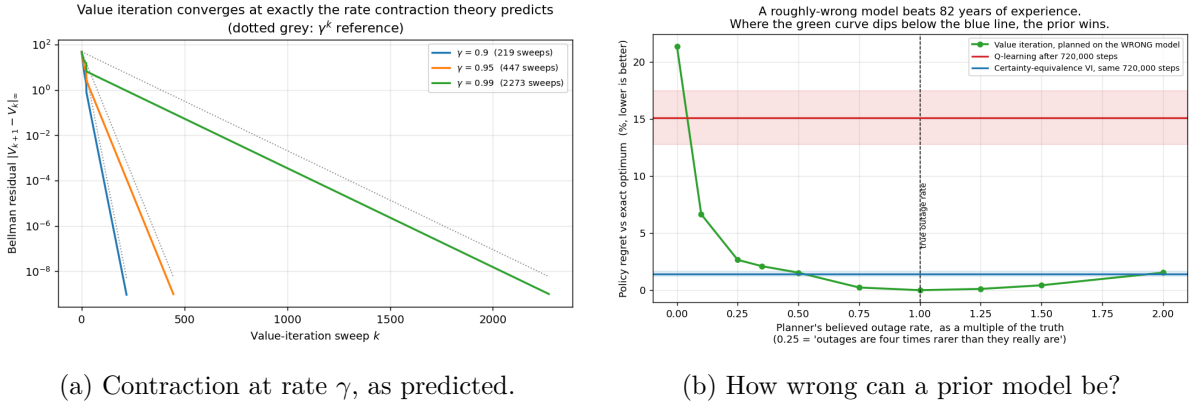


Figure 4: Assignment 3B. Right: the green curve is Value Iteration planned on a wrong model and scored in the real hall. It stays below Q-learning's red band until the planner believes the grid essentially never fails. The blue line is a model estimated from Q-learning's own samples.

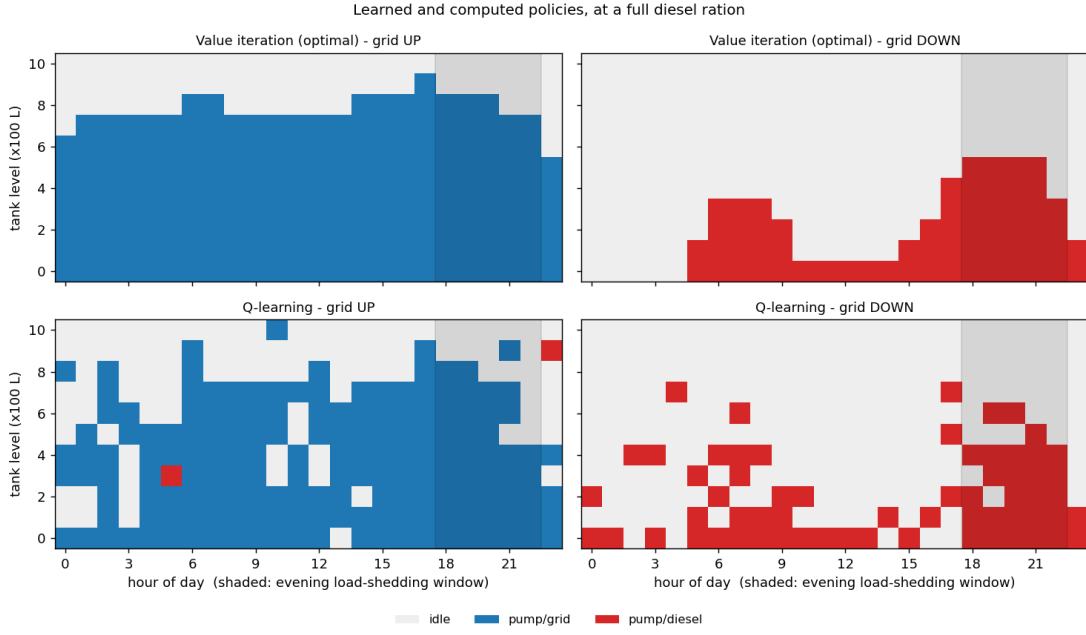


Figure 5: Policies at a full diesel ration. Value Iteration (top) is clean and shows the diesel region expanding into the shaded evening outage window — that is anticipation. Q-learning (bottom) recovers the same structure but speckled, which is what an under-trained tabular learner looks like. It picks an optimal action in 86.6% of states.

Hyperparameters. Table 6 sweeps Q-learning’s knobs at 8,000 episodes and 3 seeds. Two findings and one non-finding.

The two that matter are both large. Turning off exploring starts triples regret (76.7% against 25.1%): without random resets the agent simply never sees much of the state space. And the learning-rate schedule behaves exactly as Even-Dar and Mansour [5] says it must — the Robbins–Monro polynomial rate reaches 25.1% while a constant $\alpha = 0.1$ stalls at 50.4% and $\alpha = 0.5$ at 57.9%. A constant step size converges only to a neighbourhood of Q^* , and here you can watch it happen.

The non-finding is exploration. Varying the ε floor from 0.01 to 0.05 to 0.20 gives 30.6%, 25.1% and 27.5%, non-monotonic and inside the seed noise. We tried values from one end of the sensible range to the other and it did not much matter, so that is what we report.

Two results surprised us and we do not want to bury them. Pessimistic initialisation ($Q_0 = -200$) does slightly *better* than the zero initialisation (21.1% against 25.1%), which is the opposite of what we predicted — we had expected zero-init to be helpfully optimistic, since every reward in this MDP is non-positive. The likely explanation is scale: V^* is around -164 , so $Q_0 = -200$ starts far closer to the truth than $Q_0 = 0$ does, and that head start outweighs the lost optimism. Similarly, training with $\gamma = 0.9$ produces a policy that scores *better* (19.5%) than training with $\gamma = 0.99$ (25.1%) even when both are judged under the $\gamma = 0.99$ criterion. A shorter training horizon means lower variance and faster credit assignment, and here that is worth more than matching the evaluation discount. We chose $\gamma = 0.99$ on principle, and the data says the principle cost us something.

Table 6: Assignment 3B hyperparameter sweep. Mean regret over 3 seeds, 8,000 episodes each.

Group	Setting	Regret %
Learning rate	polynomial $1/(1+n)^{0.7}$	25.1 ± 2.1
	polynomial $1/(1+n)^{0.5}$	28.2 ± 5.9
	constant $\alpha = 0.10$	50.4 ± 2.9
	constant $\alpha = 0.50$	57.9 ± 3.8
Exploration	ε floor 0.01	30.6 ± 5.7
	ε floor 0.05	25.1 ± 2.1
	ε floor 0.20	27.5 ± 5.3
	exploring starts off	76.7 ± 7.0
Initialisation	$Q_0 = 0$ (optimistic here)	25.1 ± 2.1
	$Q_0 = -200$ (pessimistic)	21.1 ± 1.3
Discount	$\gamma = 0.90$	19.5 ± 1.4
	$\gamma = 0.95$	25.8 ± 1.4
	$\gamma = 0.99$	25.1 ± 2.1

Limitations we know about. Our grid model is a two-state Markov chain, which makes outage durations geometric: an outage that has already run four hours is exactly as likely to end as one that just started. Real Dhaka load-shedding is largely *scheduled* and rotational, so durations are far more predictable than that. Modelling the true environment as semi-Markov while the planner keeps the two-state chain would be *structural* mis-specification, which cannot be fixed by re-estimating a parameter, and we suspect that is the regime where model-free learning would finally have a real story to tell. We did not do it, and we think it is the single most interesting thing left on the table.

We also chose the discounted criterion rather than average reward. For a continuing operations problem, average cost per hour is arguably the more natural objective, and the discounted-optimal policy is not guaranteed to be gain-optimal. We report discounted return consistently everywhere and never draw a discounted V^* line on an average-cost axis, but the choice is a choice.

6. Conclusions

Four things we take away.

An admissible heuristic is easy to construct and hard to make tight. In Assignment 1 our heuristic is provably admissible and buys only an 11% reduction in nodes expanded, because on a twelve-factor multiplicative cost model the "best possible cost per metre" bound is extremely loose. Greedy Best-First, which abandons the guarantee entirely, is 79 times cheaper and 38% worse. The guarantee and its price are both real.

Propagation is what makes constraint search tractable. Forward checking with MRV cut nodes expanded by 93% at $N = 50$. But we also learned that our objective confounds solution quality with coverage at large N , which made plain MRV look worse than no heuristic at all. Designing the metric is part of designing the experiment, and we got that wrong the first time.

The swarm's advantage is the communication, not the population. This is the result we are most confident in, because we tried hard to destroy it. Thirty particles that do not share are worth no more than random sampling; the same thirty sharing a single number gain 2.47 points and connect every room. That is the assignment's thesis, and it survived our attempt to break it.

And the most useful thing we learned is a negative result. We expected to show that model-free Q-learning beats a planner working from Dhaka's optimistic load-shedding schedule. It does not. A planner whose outage estimate is wrong by a factor of four still beats a Q-learner given 720,000 steps, and a planner who is only mildly wrong beats even the model we estimated from those same samples. The real argument for model-free control is not that models are often wrong, because they can be quite wrong and still win. It is that sometimes you cannot write one down at all. Our problem was small enough to write down, so it is an honest advertisement for Value Iteration and a poor one for Q-learning. We would rather report that than the story we went looking for.

Future work, in the order we would actually do it: make the load-shedding process semi-Markov so that the planner's state is not even Markov with respect to reality; tighten the Assignment 1 heuristic; and separate quality from coverage in the Assignment 2 objective.

References

- [1] Richard Bellman. A Markovian decision process. *Journal of Mathematics and Mechanics*, 6(4):679–684, 1957. doi: 10.1512/iumj.1957.6.56038.
- [2] Geoff Boeing. OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 65:126–139, 2017. doi: 10.1016/j.compenvurbsys.2017.05.004.
- [3] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959. doi: 10.1007/BF01386390.
- [4] Marco Dorigo, Vittorio Maniezzo, and Alberto Coloni. Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 26(1):29–41, 1996. doi: 10.1109/3477.484436.
- [5] Eyal Even-Dar and Yishay Mansour. Learning rates for Q-learning. *Journal of Machine Learning Research*, 5:1–25, 2003.
- [6] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, Reading, MA, 1989.
- [7] Gergely Hajgat6, Gy6rgy Pa6l, and B6lint Gyires-T6th. Deep reinforcement learning for real-time optimization of pumps in water distribution systems. *Journal of Water Resources Planning and Management*, 146(11):04020079, 2020. doi: 10.1061/(ASCE)WR.1943-5452.0001287.
- [8] Robert M. Haralick and Gordon L. Elliott. Increasing tree search efficiency for constraint satisfaction problems. *Artificial Intelligence*, 14(3):263–313, 1980. doi: 10.1016/0004-3702(80)90051-X.
- [9] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi: 10.1109/TSSC.1968.300136.
- [10] John H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor, MI, 1975.
- [11] Ronald A. Howard. *Dynamic Programming and Markov Processes*. The M.I.T. Press, Cambridge, MA, 1960.
- [12] Shiyuan Hu, Jinliang Gao, and Dan Zhong. Multi-agent reinforcement learning framework for real-time scheduling of pump and valve in water distribution networks. *Water Supply*, 23(7):2833–2846, 2023. doi: 10.2166/ws.2023.163.

- [13] Tommi Jaakkola, Michael I. Jordan, and Satinder P. Singh. On the convergence of stochastic iterative dynamic programming algorithms. *Neural Computation*, 6(6):1185–1201, 1994. doi: 10.1162/neco.1994.6.6.1185.
- [14] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN'95 — International Conference on Neural Networks*, volume 4, pages 1942–1948, Perth, WA, Australia, 1995. IEEE. doi: 10.1109/ICNN.1995.488968.
- [15] Alan K. Mackworth. Consistency in networks of relations. *Artificial Intelligence*, 8(1): 99–118, 1977. doi: 10.1016/0004-3702(77)90007-8.
- [16] Steven Minton, Mark D. Johnston, Andrew B. Philips, and Philip Laird. Minimizing conflicts: A heuristic repair method for constraint satisfaction and scheduling problems. *Artificial Intelligence*, 58(1–3):161–205, 1992. doi: 10.1016/0004-3702(92)90007-K.
- [17] Shengwei Pei, Lan Hoang, Guangtao Fu, and David Butler. Real-time pump scheduling in water distribution networks using deep reinforcement learning. *Journal of Water Resources Planning and Management*, 151(6):04025012, 2025. doi: 10.1061/JWRMD5.WRENG-6476.
- [18] Ira Pohl. Heuristic search viewed as path finding in a graph. *Artificial Intelligence*, 1(3–4): 193–204, 1970. doi: 10.1016/0004-3702(70)90007-X.
- [19] Theodore S. Rappaport. Wireless communications: Principles and practice. *Prentice Hall*, 2002.
- [20] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, Hoboken, NJ, 4th edition, 2021. ISBN 9780134610993.
- [21] Yuhui Shi and Russell C. Eberhart. A modified particle swarm optimizer. In *Proceedings of the 1998 IEEE International Conference on Evolutionary Computation*, pages 69–73, Anchorage, AK, USA, 1998. IEEE. doi: 10.1109/ICEC.1998.699146.
- [22] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, MA, 2nd edition, 2018. ISBN 9780262039246.
- [23] John N. Tsitsiklis. Asynchronous stochastic approximation and Q-learning. *Machine Learning*, 16(3):185–202, 1994. doi: 10.1007/BF00993306.
- [24] Christopher J. C. H. Watkins. *Learning from Delayed Rewards*. PhD thesis, King’s College, University of Cambridge, Cambridge, UK, 1989.
- [25] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4): 279–292, 1992. doi: 10.1007/BF00992698.
- [26] David H. Wolpert and William G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997. doi: 10.1109/4235.585893.